OS16 - Project Concept, Model and Initial Forecast

Team 14A: David Ricardo, Camden Ramsay, Nora Alshareef, Michel Fernandes

20 April 2026

Sanguine is a physician-facing clinical interpretation layer for Mayo Clinic laboratory results. It aims to improve Mayo Clinic lab patient care outcomes at scale by augmenting physician-facing test results with relevant research using an agentic AI clinical librarian.

1: By-Hand Project Concept, Targets, & Elements

1.1: Targets

This table shows our updated targets. Appendix 1 shows how they've evolved since DC2 Project Charter.

Dimension	Metric	Target	Rationale
Scope	Test types supported at full launch	3 types	"Metabolic panel" is one type, "blood cell count" is another, and so on. Three types cover the most common tests. Tradeoff: include esoteric tests that aren't given very often but where sanguine could give high value per test result, or aim for common tests with lots of results, even though doctors would need sanguine less for those?
	Accuracy of suggestions	98% on feedback	Accuracy is our most important metric, so to prioritize this, we set weaker targets on the other dimensions of scope.
	P99 added latency	<60 minutes	Most tests are not critical, so delaying results by this many minutes is acceptable.
	Jurisdictions approved	50 US states	This determines the possible user base. In this first iteration, we do not need legal approval for countries outside the US.
Schedule	Development start date	June 2026	Right after our presentations in mid-May, plus a couple of weeks to arrange funding.
	First prototype ready to test	Q4 2026	This ensures the project demonstrates a deliverable in 2026 and burns down technical risk on the most complex area. This is feasible because the scope of the prototype is small.
	Private pilot launch	Q1 2027	Launching an MVP with one test type to 1-10 partner providers lets us validate and collect early feedback.
	Public initial launch date	Q3 2027	Begin with a US launch to keep customers in the same time zone and under a single federal regulatory framework.
	Internationalization expansion plan complete	Q4 2027	Sign-offs on an expansion plan are a major milestone.
	Global launch date	Q1 2028 - Q3 2027	Global launch is required because many of the most valuable tests are for people in other countries. This has a range that will be tightened in the internationalization plan. Challenges here will be scale, refinement to local expectations, and regulatory compliance.
Budget	Initial costs	<$6M	This includes both one-time engineering costs and capital costs. They are combined into one line item to give the project manager flexibility for how to divide. Our budget target was $30M in IAP, but as we've increased our understanding and decreased the scope, the required budget lowers.
	Annual fixed run-rate cost	<$1M	Ongoing costs for maintenance, infrastructure, and support - costs that are not linked to tests.
	Total delivery cost per test	<$10	This must not materially impact our test delivery cost.

1.2: Deliverables in Product Breakdown Structure

- Design Ready
  - Requirements approved (David note: maybe not in scope? We'll see what they say on OS15)
  - High-level design approved
  - Low-level designs approved
- Pilot launch
  - Interpretation prototype complete
  - Designs adjusted to incorporate prototype lessons
  - Core pathway code complete
  - Full verification of code meets requirements
  - Pilot launch
  - Validation with partner feedback
- US launch
  - Formal legal approval
- Full global launch
  - Internationalization expansion plan approved (which will include deciding which regions to expand to first, which have overlapping requirements, etc.)
  - Phased approach: Launch in region X
  - Launch in region Y
  - Global launch date

1.3: Phases in Work Breakdown Structure

The work breakdown structure (WBS) illustrates the phases and work to be completed during each phase. There is a specified phase gate exit item. For a more expansive WBS including one additional layer, please see Appendix 5: Expanded WBS.

1. System Architecture and Definition Phase:
   - 1.1. Requirements Definition
   - 1.2. Data Interface & Harmonization Design
   - 1.3. Multi-Agent Architecture Definition
   - 1.4. Output Specification
   - **1.5. System Architecture and Definition Phase Gate Exit**

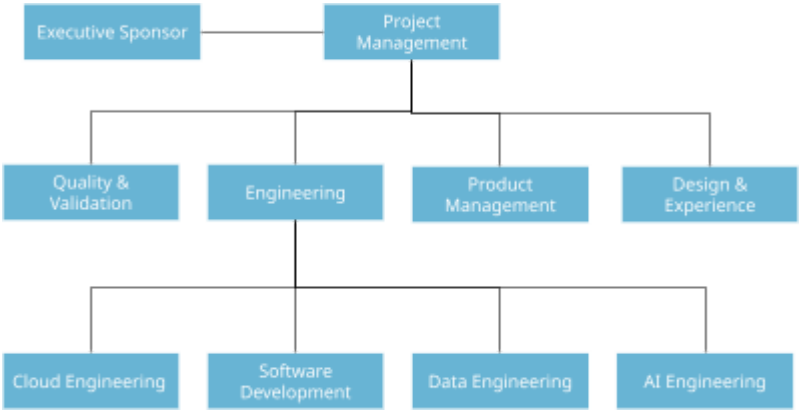
2. Component Engineering and Development Phase
   - 2.1. Data Context & Retrieval Pipeline
   - 2.2. Agentic Core Development
   - 2.3. Physician-Facing Output Interface
   - **2.4. Component Engineering and Development Phase Gate Exit**

3. Integration and V&V Phase:
   - 3.1. Component & System Integration Testing
   - 3.2. Exception & Resilience Testing
   - 3.3. Shadow-Mode Clinical Validation
   - 3.4. Prototype Launch & A/B Evaluation
   - **3.5. Integration and V&V Phase Gate Exit**

4. Operational Phase:
   - 4.1. Regulatory & Compliance Readiness
   - **4.2. Product Launch Gate Exit**
   - 4.3. Private Pilot Launch
   - 4.4. US Public Launch
   - 4.5. Internationalization & Global Launch

1.4: Teams in Org Breakdown Structure

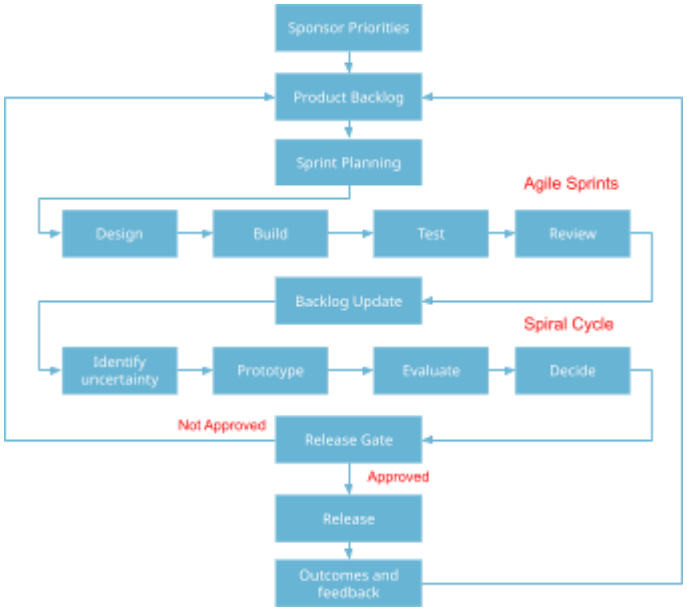
The following teams constitute the full Organization Breakdown Structure (OBS). Each team maps to a PDQA role classification: Product (P), Delivery/Engineering (D), Quality (Q). Rates are sourced from the BLS Occupational Outlook Handbook (2025) [1]-[6] and reflect median US market compensation. Overhead accounts for benefits, facilities, tooling, and administrative burden.



Team	PDQA	Rate	Overhead	Required Resources
Cloud Engineering	P/D	$50/hr	+30%	Cloud infrastructure for development, testing, and production deployment
Software Development	D	$64/hr	+50%	Development environment; CI/CD pipeline infrastructure
AI Engineering	D	$54/hr	+20%	LLM API access; GPU compute for model evaluation and testing
Data Engineering	D	$65/hr	+50%	Internal Mayo clinical data sources; licensed external clinical literature
Quality Assurance	Q	$50/hr	+50%	All system artifacts; test environment access; clinical validation datasets
Product Management	P	$48/hr	+50%	All solution artifacts; stakeholder access; roadmap tooling
Design & Experience	D	$47/hr	+50%	Output channel artifacts; physician UX research access

Allocation Pattern

Allocation follows a demand-driven model tied to WBS phase activity. AI Engineering and Data Engineering carry the highest headcount during Phase 2 (Component Engineering), where core pipeline and agentic development is concentrated. Quality Assurance scales up during Phase 3 (V&V) as test coverage expands. Product Management and Design & Experience maintain lighter but consistent involvement across all phases to ensure alignment with physician needs and product direction. Cloud Engineering is lean throughout but spikes during integration and launch milestones. This pattern reflects the hybrid Agile-Spiral concept: sustained parallel delivery tracks with coordinated surge capacity at key gates.



1.5: Project Concept Diagram

This project combines elements of waterfall, agile, and spiral to get the benefits of each. At a high level, we use waterfall-like phase discipline through a few stage gates to keep the project on track to major milestones. At a low level, during implementation, an agile approach provides the main delivery structure through a backlog, sprint development, and continuous integration across all teams (design, engineering, and testing). The spiral elements were introduced around parts of the system with high uncertainty, where activities of prototyping, validation, and risk reduction must occur before release decisions.

2: Project Model

Model Development

We implemented our project model from scratch in Python in Google Colab. We built an agent-based framework centering on Teams and Tasks. We also implemented Phases, Products, Productivity multipliers (such as overtime), even risks to set ourselves up for OS17. We wrote the code by hand with plenty of comments and minimal AI assistance. It is publicly viewable at ∞ OS16 Colab, and is detailed in Appendix 2: Model Description.

Model Forecasts

The following are the 10 iterations of our model and their respective baseline predictions. The baseline consists only of internal resources, and then we tested adjustments to capacity, sequencing, a small change in scope, and coordination, recording the total cost and schedule in weeks. Each line describes the change applied to the model and the observed trade-off, allowing for consistent comparison of alternatives against the same reference.

#	Model change	Cost	Schedule	Commentary
1	Initial model	$4,798,352	46 weeks	Original reference model, based on internal resources only.
2	Outsource development	$3,728,408	55 weeks	Lower labor rates via contractors, but with high coordination overhead.
3	Parallelize launches	$5,319,912	51 weeks	Remove blockers between launches.
4	Stable core team	$5,591,388	44 weeks	Increases steady staffing cost with improving continuity.
5	Overtime development	$6,237,692	46 weeks	Aggressive overtime with extra costs.
6	Grow workforce	$7,719,088	37 weeks	Scale workforce to support all phases.
7	Dedicated integration team	$5,265,568	44 weeks	Adds a specialized coordination to integration-heavy phases, improving flow.
8	Front-load integration	$4,902,664	47 weeks	Adds earlier coordination efforts to reduce downstream interface rework.
9	Prebuilt solutions	$4,862,968	44 weeks	Reduces internal engineering effort but increases integration dependency.
10	Strategic scaling	$4,692,776	43 weeks	Hybrid approach using contractors for non-critical development paths.

Strategic Decisions

Decision 1: Workforce sourcing and scaling strategy

By using contractors, we were able to reduce direct labor costs and gain the ease to grow when necessary. The trade-off is that when outsourcing moves into core functions, coordination effort increases and decision alignment becomes more difficult, especially at interfaces, due to a loss of context, more frequent handoffs, and the significant turnover (13%) [7]. This is clear in the "Outsourced Development" variant, where the cost decreased (-22%), and the schedule was extended (+19%) at the same level. Using a hybrid approach, as the "Strategic Scaling" variant, contractors were used only for less critical work, while preserving the core functions and integration with internal teams. Compared to the baseline, this projected the schedule down (-6%), with less impact on total cost (-2%).

Decision 2: Make-buy solution strategy

The make-buy decision determines whether Sanguine's core AI components are built internally, sourced from existing commercial and open-source platforms (e.g., LangChain, Pinecone, etc.) or a mix of both. The options range from full internal build, maximizing control but requiring significant engineering effort, full reliance on prebuilt platforms, to a hybrid approach where commodity infrastructure is purchased while clinically-specific interpretation logic is built in-house.

The use of prebuilt components aims to reduce direct engineering effort and accelerate delivery by transferring construction complexity to integration and external supplier management. This decision is especially significant for Sanguine because the 98% accuracy requirement and Mayo Clinic integration constraints mean that core

interpretation logic cannot be delegated to a generic platform, getting this boundary wrong risks under-investing in the parts that determine clinical value. The trade-off is that integration and V&V become more complex, which can lead to rework in earlier project stages.

The "Prebuilt solutions" variant captures this by reducing construction effort on T2.1 (Data Context and Retrieval Pipeline), T2.2 (Agentic Core Development), and T2.3 (Physician Facing Output Interface) by 30%, while increasing T1.2 (Data Interface and Harmonization Design) and T3.1 (Component and System Integration Testing) by 25% to reflect integration overhead. Compared to the baseline, the schedule improved (−4%) and total cost increased slightly (+1%), suggesting a selective buy strategy for non-core components is worth pursuing while internal build remains preferable for the clinically sensitive agent logic.

Decision 3: Integration and coordination structure

The integration structure decision determines how Sanguine's parallel development tracks, the data pipeline, agentic core, and physician-facing interface are coordinated across teams to avoid late-stage interface failures. The options are: (1) Sequential phase gating: integration begins only after all components are complete, concentrating interface risk at the end; (2) Dedicated integration team: a standing cross-functional team owns interface validation throughout the project; (3) Front-loaded integration: integration checkpoints run partially in parallel with component development to surface issues earlier.

This decision is especially significant for Sanguine because the system has three independently developed AI agents (hematology, nephrology, and hepatic interpreters) that must each interface correctly with the shared orchestration layer, the data pipeline, and the physician UI. Interface failures discovered late, after each component has been validated in isolation, are among the most expensive and schedule-threatening problems in AI system integration. The clinical validation requirements compound this risk: a misaligned agent output that passes unit tests but fails in shadow-mode clinical review requires rework across multiple components simultaneously.

The model captures both alternatives. The "Dedicated integration team" variant adds a four-person team to T3.1 (Component and System Integration Testing), T3.2 (Exception and Resilience Testing), and T3.3 (Shadow Mode Clinical Validation) with a 0.95 productivity multiplier reflecting coordination overhead, yielding $5.27M / 44 weeks (+10% cost, stable schedule). The "Front-load integration" variant adds earlier dependency links between development and integration tasks, with a 1.08 learning multiplier activating after week 16, yielding $4.90M / 47 weeks. Front-loading preserves cost but extends the schedule, while a dedicated team costs more but protects the timeline. Given Sanguine's Q4 2026 prototype target, the dedicated integration team is the more defensible choice.

Decision 4: Schedule compression

The schedule compression decision determines how the team should respond if delivery milestones are at risk, specifically, whether to pursue overtime, full headcount growth, or targeted contractor augmentation on bottleneck tasks to accelerate the schedule. The options are: (1) Overtime, apply a productivity boost during a defined sprint window at 1.5 pay, accepting the risk of subsequent burnout and later productivity drop, aligned with Brooks's point that extra efforts doesn't convert directly into schedule gains [8]; (2) Grow workforce, double team sizes across all functions to maximize parallelism, accepting onboarding friction in early weeks; (3) Strategic scaling, add contractor capacity selectively to the non-critical development paths, minimizing coordination overhead while targeting the tasks that aren't from core functions

This decision is critical because Sanguine has a hard external milestone: a working prototype must be ready by Q4 2026, roughly 28 weeks after the June 2026 development start. Any compression strategy must therefore deliver real schedule improvement, not just increased cost. Overtime in particular carries a well-documented risk of generating the illusion of acceleration while producing burnout-driven slowdowns shortly after, a pattern that is especially dangerous in a clinical AI system where accuracy and code quality cannot be traded off.

The model makes this tradeoff quantitatively visible. The "Overtime development" variant applies a 1.60 productivity multiplier between weeks 20–28, followed by a 0.70 burnout multiplier through week 36, with all team costs increased by 30%, yielding $6.24M / 46 weeks, identical schedule to baseline at 30% higher cost, confirming that overtime does not compress the schedule when burnout is modeled. "Grow workforce" delivers 37 weeks at $7.72M, a 60% cost premium. "Strategic Scaling" adds six contractors only to T2.1 (Data Context and Retrieval Pipeline), T2.2 (Agentic Core Development), and T2.3 (Physician Facing Output Interface) with a 0.85 friction multiplier, achieving 43 weeks at $4.69M, saving 3 weeks while reducing cost by 2% relative to baseline, making it the strongly preferred compression strategy.

3. Forecast and Initial Tradespace

The project model was used to generate nine variants from the baseline (a total of ten variants) by adjusting teams, scope, and architecture across four strategic decisions. Each variant was simulated week-by-week and the results tracked in the appendix table. The exploration was structured around the four strategic decisions identified in Q2, with each variant isolating a specific change to allow clean comparison against the baseline.

Of the ten variants generated, seven involve changes to teams/resources (“Outsource development”, “Strategic Scaling”, “Dedicated integration team”, “Stable core team”, “Grow workforce”, and “Overtime development”, plus the internal-only baseline as the reference configuration), one involve changes to scope (“Prebuilt solutions”), and two involves changes to sequencing and dependencies (“Front-load integration” and “Parallelize launches”, which restructure when integration and launch activities can proceed). The majority of variants are therefore resource-driven, reflecting that for a software-intensive AI project, staffing decisions dominate both cost and schedule outcomes.

The exploration revealed three key insights. First, workforce sourcing has the highest sensitivity across all decisions. Moving from full outsourcing to strategic scaling alone shifts the schedule by 12 weeks ($3.7M / 55 weeks vs. $4.7M / 43 weeks), confirming that how and where contractors are introduced matters more than whether they are used at all. Full workforce growth achieves the fastest schedule at 37 weeks, but at a 60% cost premium over baseline, a tradeoff that is difficult to justify given that the $6M budget target already provides 22% headroom under strategic scaling. This suggests that contractor augmentation on bottleneck tasks is the right lever, not blanket headcount growth.

Second, schedule compression through overtime is ineffective when burnout is modeled honestly. The overtime variant applies a 1.60 productivity multiplier during weeks 20-28, but this is followed by a 0.70 burnout multiplier through week 36. The net result is $6.24M / 46 weeks, identical schedule to baseline at 30% higher cost. This was one of the more counterintuitive findings from the model, and it led the team to rule out overtime as a compression strategy entirely. It also highlighted an important modeling principle: short-term productivity gains must always be evaluated against their downstream recovery cost.

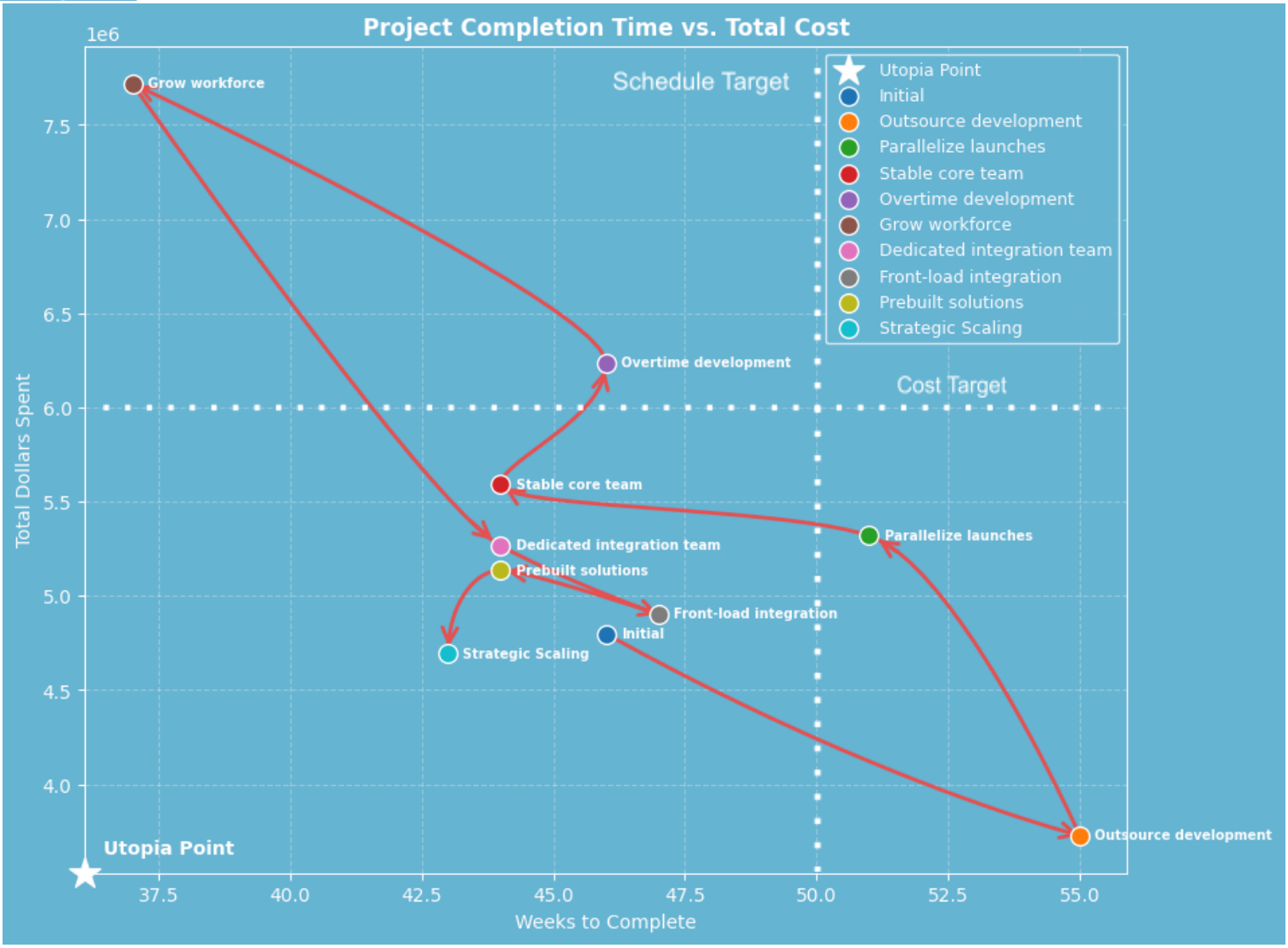
Third, the integration structure affects cost more than schedule. Both the dedicated integration team ($5.27M / 44 weeks) and front-load integration ($4.90M / 47 weeks) variants land within a narrow schedule range of 44-47 weeks, but differ by $370K in cost. This suggests that for Sanguine, integration decisions are primarily a cost lever rather than a time lever. The dedicated integration team is preferred because it protects the Q4 2026 prototype deadline without relying on schedule recovery later.

One limitation identified during modeling is that the model does not currently capture cascading rework risk. If a clinical validation failure in Phase 3 requires changes to the agent logic developed in Phase 2, the additional effort and schedule impact of that rework loop is not represented in the current model. This is a meaningful gap for a clinical AI system where shadow-mode validation failures are a realistic and potentially high-impact event. This is flagged as a priority gap to address in OS17 through explicit risk modeling, potentially using the rework risk infrastructure that is already built into the simulation engine but not yet applied to the Sanguine project tasks.

A second modeling gap identified is that productivity multipliers are currently applied uniformly across all tasks assigned to a team, rather than being task-specific. In practice, a contractor added to support T2.1 (Data Context and Retrieval Pipeline) would not experience the same coordination friction on T2.2 (Agentic Core Development) since they may not be assigned to both. Future iterations of the model could improve realism by making friction multipliers task-scoped rather than team-scoped.

4. Tradespace Diagram, Baseline, and Insights

Diagram



Evolution

The project design evolved through three stages of exploration. First, the initial baseline assumed full internal staffing, sequential phase gating, and an entirely built-from-scratch stack, yielding $4.80M / 46 weeks. The second round of exploration tested extreme options: full outsourcing, full workforce doubling, and overtime, to establish the boundaries of the tradespace. These extremes confirmed that aggressive interventions either extend the schedule significantly (outsourcing, +19%) or carry unsustainable cost premiums (workforce growth, +60%, overtime, +30%) without proportional schedule benefit.

The third round focused on surgical interventions: strategic scaling, prebuilt solutions, dedicated integration team, and front-loading, to find options that improve both dimensions simultaneously or make targeted tradeoffs. This round revealed that Strategic Scaling is the only variant that improves both cost and schedule relative to baseline, making it the dominant option on the Pareto frontier closest to the Utopia Point.

Initial Plan

The team's preferred baseline is **Strategic Scaling** at $4.69M / 43 weeks. It satisfies the $6M budget target with 22% headroom, achieves the Q4 2026 prototype milestone comfortably, and avoids the coordination risks of full outsourcing. It is the closest point to the Utopia Point on the tradespace diagram, meaning no other tested architecture is both cheaper and faster simultaneously. This plan will serve as the baseline carried forward into OS17.

Appendix 1: Target Comparison

To show our target evolution, this table shows our charter targets from DC2 Project Charter in January, with commentary on each row and whether each target is included in Q1 above. We've learned a lot since then!

Dimension	Previous Target (Desired)	Previous Acceptable Range	Previous notes	Commentary	Kept?
Scope / Performance	Feasibility assessment across 3 AI capability areas	All 3 required	EMR integration, analytics, and communication	Irrelevant. Feasibility is part of the planning process, not part of the delivered scope.	❌
	Regulatory/compliance risk identification and mitigation	—	In the US, this means HIPAA, CLIA, and approval by the FDA and CAP if necessary. Globally, others apply.	Still in scope, but refined. Now, our metric is "jurisdictions".	✅
	Implementation roadmap with phasing options	Minimum 2 options	Build vs. buy analysis	Still relevant, but analyses like this are part of planning, not delivered scope. Ironically, this foreshadowed the decisions in OS16 Q2!	❌
	Applies to 5 different test types where it's needed most	4-6	"Metabolic panel" counts as one type, "blood cell count" is another, and so on.	Still a good metric.	✅
Schedule	Planning complete by May 2026	Non-negotiable	MIT SDM program deadline	We misunderstood the boundary between the EM.413 scope and our project scope.	❌
	Pilot program launched to trusted physicians only by September 2026	Anytime in September	To declare an initial launch in Q3.	Refined based on feedback on our previous charter.	✅
	MVP launch: improving one test type for patients in one regulatory environment	January - February 2027	Launching anything in Q4 is hard, so we can wait until 2027.	Refined based on feedback on our previous charter.	✅
	Global solution at scale	Q3 2027	To complete within fiscal year 2027	De-scoped to just US to ease legal approvals.	✅
Cost	MIT SDM team effort ~200 hours	+20% flex	4 team members × ~50 hrs each	We misunderstood the boundary between the EM.413 scope and our project scope.	❌
	Mayo staff time ~20 hours	+10 hours	Interviews + document review	We misunderstood the boundary between the EM.413 scope and our project scope.	❌
	Capital expenditure $0	$0	Should leverage existing capital	Incorporated into "one-time costs" below.	❌
	Software engineering cost (one-time)	$10M - $30M	Mayo would be happy with any budget within this wide range because cost is our lowest priority.	Decreased massively (and removed the bottom of the range) based on learnings since then.	✅
	Cloud expenses (monthly)	$0.2M - $0.5M	This is low because of Mayo's investments.	Cost per test is a better metric as our deployment scales.	❌

Appendix 2: Model Description

This appendix is in three parts: the project modeling framework we built, the framework's exclusions, and our model within that framework. Our code is world-readable at OS16 Colab, with many comments and unit tests. Our model's design goals are to be as insightful as possible while matching the structure of OS16 and OS17.

Modeling Framework

We designed this framework to natively model our project as broken down in Q1. A `Project` is defined by a list of each of these types. Each type also has an ID and name for tracking.

- A `Team` has a number of people, cost per person per week, and employment type (full-time or contractor). This models our strategic in-house vs outsource decision. Contractors bill for work performed, while full-time employees need salaries even when blocked.
- A `Phase` and `Product` have no effect on the simulation, but are used for insights.
- A `Task` has effort in person-weeks, a maximum parallelism in number of people, a set of teams that could work on this task, a set of blocking tasks, a non-labor cost, and a list of associated `Phases` and `Products`.
- A `ProductivityMultiplier` affects productivity of a team temporarily, such as for overtime.

Each `Project` is simulated in a deterministic `Simulation`. Within a `Simulation`, a `ModelSnapshot` for each week captures state that varies over time, such as money spent and the effort remaining on each `Task`. We also implemented a `DesignWalk`, where each point in the design walk is defined by a function on a previous project.

Intentional Exclusions

All models are wrong, but some models are useful. These exclusions are how our modeling framework is wrong, and why it is still useful.

- We assume each (non-rework) task's effort and dependencies are known in advance. This simplifying, Taylorist assumption aligns with the high level waterfall-like structure with stage-gate validations.
- We do not model coordination overhead within a team or across teams. Each task has a `max_parallelism`, but is parallelizable perfectly below that limit. This assumption models our agile style within each stage gate. We considered adding more detail here, but we thought the in-class spreadsheet macro activity was insightful even with this assumption, so we kept it here too.
- We do not model each team's location. In a software project where remote work is common, we believe modeling each team's time zone would add more complexity than insight. We include productivity allocations that we negatively penalize for communication or timezone barriers that allow for some localization insights.
- We model that each team finishes tasks first-in-first-out, and when multiple tasks start at the same time, teams divide headcount evenly among them. We needed to choose some allocation method.
- Each task has a `non_labor_cost`, but we assume no competition for non-labor resources.
- We implemented risks, but felt scope was better served if we reserved them for OS17.

Project Model Description

Because we designed our library to mirror the structure of Q1, the Sanguine project appears almost exactly as it does above in Q1. Our initial model is viewable in this cell. For example, task 1.2 "Data Interface & Harmonization Design" appears in our model with the same name and identifier. We planned sub-tasks within those, as shown in Appendix 4, but did not implement them because we found our project exploration was still insightful without them.

Our model of Sanguine has seven `Teams`, seven `Products`, and 21 `Tasks`. Even though most of the development day-to-day is agile or spiral, as discussed in Q1.4, the waterfall-style stage gates are represented in a high-level structure. For choosing which tasks blocked which other tasks within a phase, we used our intuition as a group of software engineers, and validated this assumption against our project scope, schedule, and budgets. This is an area where, as data becomes more available to the PM, updating the model with those findings will yield more accurate forward projections.

We implemented our `DesignWalk` with a series of functions that take a Project and mutate it. For example, we explored outsourcing with a function named `outsource_effort`. You can view our full design walk in this cell. Often, we found that it was best to start from the same initial station position for each variation, but have the ability to build upon the last, or any other starting position for a new variant.

Appendix 3:Table of Iterations

#	Decision Changed	Prior option	New option	Categorization	Cost	Schedule [weeks]	Commentary
1	Initial	Baseline		Teams / Resources	$4,798,352	46	Internal execution, without contractors.
Converted SWE and DATA to contractors with lower weekly cost; added persistent interface-friction productivity loss; increased effort and parallelism on T2.1–T2.3 (Data Context and Retrieval Pipeline, Agentic Core Development and Physician Facing Output Interface).							
2	Outsource development	Initial	Outsource engineering	Teams / Resources	$3,728,408	55	Lower labor rates via contractors, but high coordination overhead.
Removed launch-gate blockers to overlap launch activities; tripled launch-gate effort to reflect running multiple launches at once.							
3	Parallelize launches	Initial	Run Pilot, US, and Global launches in parallel	Sequencing / Dependencies	$5,319,912	51	Remove blockers between launch gates, but 3x launch gate effort because it's three launches at once
Increased SWE/DATA/AI/QA teams' headcount and weekly cost; added a continuity productivity boost starting after the early project period.							
4	Stable core team	Initial	Maintain a stable cross-phase core team	Teams / Resource	$5,591,388	44	Increases steady staffing cost, but improves continuity, knowledge retention, and handoff quality.
Applied a mid-project productivity boost, followed by burnout; increased weekly cost across all teams to reflect overtime premium.							
5	Overtime development	Initial	Overtime during the development phase	Teams / Resource	$5,319,912	46	Aggressive OT with 1.5x pay premium for OT hours.
Increased SWE/DATA/AI/QA teams' headcount and weekly cost; added a continuity productivity boost starting after the early project period.							
6	Grow workforce	Initial	Larger team size	Teams / Resource	$7,719,088	37	Scale workforce to support all phases.
Added a 4-person internal integration team; assigned it to T3.1–T3.3 (Component and System Integration Testing, Exception and Resilience Testing, and Shadow Mode Clinical Validation) with higher parallelism; added a small persistent coordination productivity loss for that team.							

#	Decision Changed	Prior option	New option	Categorization	Cost	Schedule [weeks]	Commentary
7	Dedicated integration team	Initial	Create a dedicated team for cross-team integration	Teams / Resource	$5,265,568	44	Adds a specialized coordination layer to integration-heavy phases, improving flow at the cost of extra staffing.
Increased SWE/DATA/AI/QA teams' headcount and weekly cost; added a continuity productivity boost starting after the early project period.							
8	Front-load integration	Initial	Move integration and interface validation earlier	Sequencing / Dependencies	$4,902,664	47	Adds earlier coordination effort, but reduces downstream interface surprises and rework.
Reduced build effort on T2.1–T2.3 (Data Context and Retrieval Pipeline, Agentic Core Development and Physician Facing Output Interface); increased effort on T1.2 (Data Interface and Harmonization Design) and T3.1 (Component and System Integration Testing); raised SWE/DATA/AI weekly cost and added persistent vendor-dependency productivity loss.							
9	Prebuilt solutions	Initial	Use prebuilt components and external platforms	Scope	$5,136,208	44	Reduces internal engineering effort, but increases integration dependency and external platform overhead.
Added a 6-person contractor support team; assigned it to T2.1–T2.3 (Data Context and Retrieval Pipeline, Agentic Core Development, and Physician Facing Output Interface) with higher parallelism; added persistent external-coordination productivity loss.							
10	Strategic Scaling	Initial	Targeted scaling for bottlenecks	Teams / Resource	$4,692,776	43	Hybrid approach using contractors for non-critical dev paths.

Note: Appendices 1-3 are specifically required to meet requirements of the assignment. Appendices 4-8 were not required by the assignment, but added considerable value in our development process and are included for context. It felt natural to create a break between these sections to add clarity to the document, which is what this note serves to accomplish.

Appendix 4: Expanded WBS

1. System Architecture and Definition Phase:

- 1.1. Requirements Definition
  - 1.1.1. Elicit and document clinical stakeholder requirements (physicians, administrators)
  - 1.1.2. Define system functional and non-functional requirements (accuracy, latency, compliance)
  - 1.1.3. Obtain requirements sign-off from Mayo Clinic sponsors
- 1.2. Data Interface & Harmonization Design
  - 1.2.1. Map EHR/LIS data schemas and identify integration points
  - 1.2.2. Define data normalization and standardization rules for lab result ingestion
  - 1.2.3. Design patient context retrieval interfaces
- 1.3. Multi-Agent Architecture Definition
  - 1.3.1. Define roles, responsibilities, and trust boundaries for each agent
  - 1.3.2. Specify orchestration patterns and inter-agent communication protocols
  - 1.3.3. Document fallback and error-handling behaviors
- 1.4. Output Specification
  - 1.4.1. Define physician-facing output format, content, and presentation standards
  - 1.4.2. Specify feedback capture mechanism design
  - 1.4.3. Establish clinical accuracy and compliance review criteria
- **1.5. System Architecture and Definition Phase Gate Exit**

2. Component Engineering and Development Phase

- 2.1. Data Context & Retrieval Pipeline
  - 2.1.1. Ingest and standardize raw laboratory results
  - 2.1.2. Integrate patient history and clinical context retrieval
  - 2.1.3. Connect and validate internal Mayo Clinic reference sources
  - 2.1.4. Integrate and validate external clinical literature sources
- 2.2. Agentic Core Development
  - 2.2.1. Build orchestration layer for multi-agent task management and auditing
  - 2.2.2. Develop context retrieval and evidence ranking agent
  - 2.2.3. Implement hematology interpreter (complete blood count panel)
  - 2.2.4. Implement nephrology interpreter (comprehensive metabolic panel)
  - 2.2.5. Implement hepatic interpreter (liver function panel)
  - 2.2.6. Build clinical reconciliation and care-model alignment agent
  - 2.2.7. Implement compliance and safety guardrail agent
- 2.3. Physician-Facing Output Interface
  - 2.3.1. Develop output rendering and delivery channel
  - 2.3.2. Build structured physician feedback and rating mechanism
- **2.4. Component Engineering and Development Phase Gate Exit**

3. Integration and V&V Phase:

- 3.1. Component & System Integration Testing
  - 3.1.1. Verify inter-component interfaces and data flow
  - 3.1.2. Execute system-level integration test suite
- 3.2. Exception & Resilience Testing
  - 3.2.1. Conduct exception injection to validate failure handling and graceful degradation
  - 3.2.2. Test edge cases: ambiguous results, missing context, out-of-range values

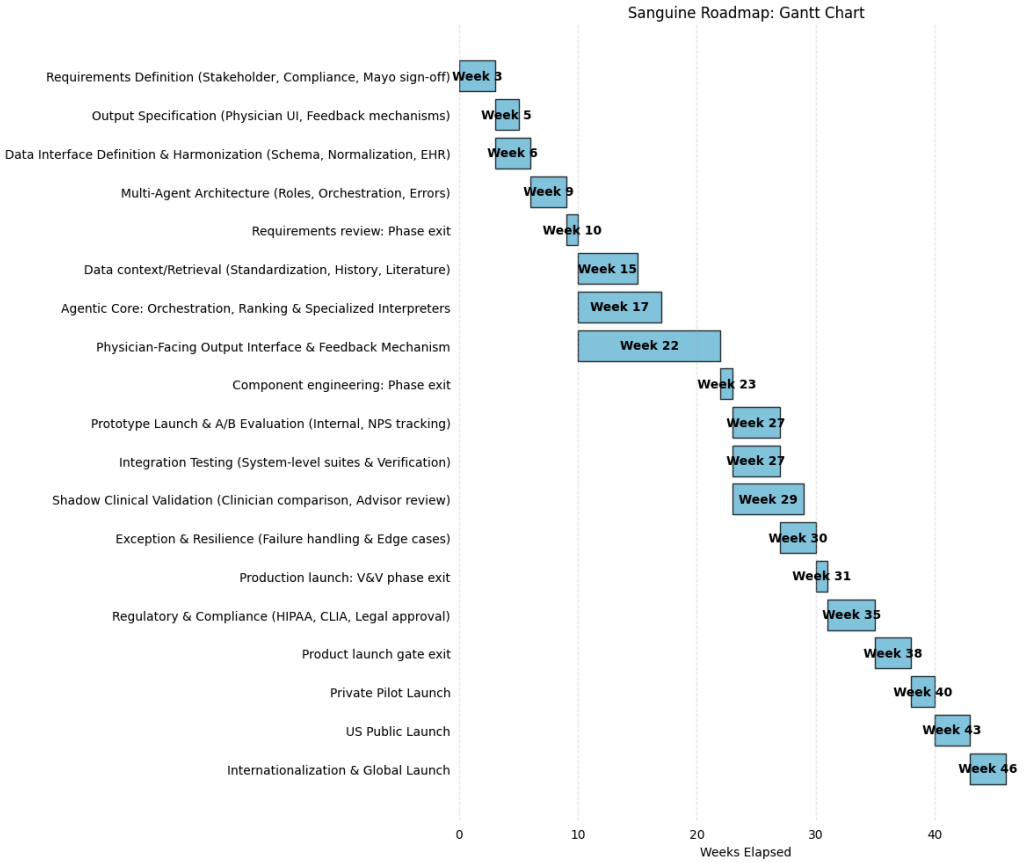
- 3.3. Shadow-Mode Clinical Validation
  - 3.3.1. Deploy system in shadow mode alongside current clinical workflow
  - 3.3.2. Compare agentic outputs against clinician decisions; measure divergence rate
  - 3.3.3. Conduct clinical and data coherence review with Mayo Clinic advisors
- 3.4. Prototype Launch & A/B Evaluation
  - 3.4.1. Deploy prototype to controlled internal environment
  - 3.4.2. Run A/B variants to optimize for physician engagement, NPS, and alert fatigue reduction
  - 3.4.3. Incorporate findings into pre-launch refinements
- **3.5. Integration and V&V Phase Gate Exit**

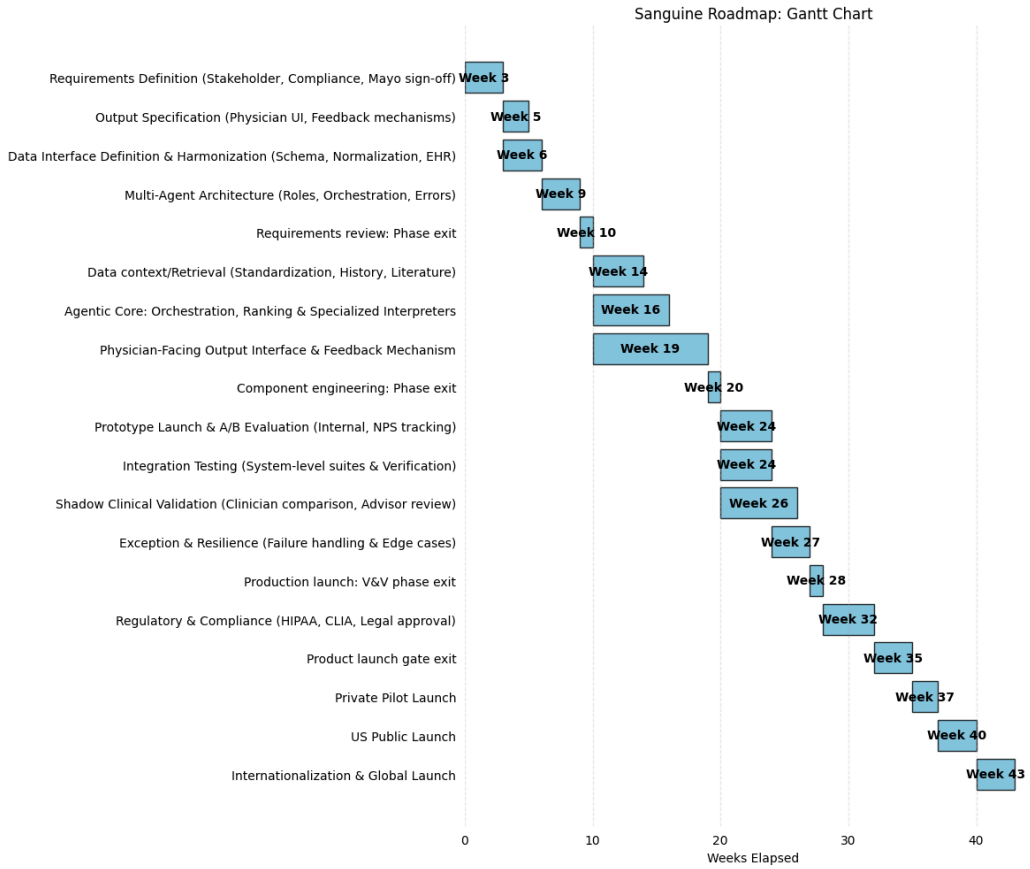
4. Operational Phase:

- 4.1. Regulatory & Compliance Readiness
  - 4.1.1. Complete HIPAA, CLIA, and applicable FDA/CAP documentation
  - 4.1.2. Obtain formal legal approval for US operations across all 50 states
  - 4.1.3. Establish ongoing compliance monitoring and audit process
- **4.2. Product Launch Gate Exit**
- 4.3. Private Pilot Launch
  - 4.3.1. Onboard 1–10 partner providers; configure for single test type
  - 4.3.2. Collect structured feedback; track accuracy, latency, and physician NPS
  - 4.3.3. Iterate on model and interface based on pilot findings
- 4.4. US Public Launch
  - 4.4.1. Scale infrastructure to support full US deployment
  - 4.4.2. Expand to three supported test types
  - 4.4.3. Activate production monitoring: model drift, latency, feedback rates, incident response
- 4.5. Internationalization & Global Launch
  - 4.5.1. Develop and obtain approval for internationalization expansion plan
  - 4.5.2. Adapt system for regional regulatory regimes and clinical expectations
  - 4.5.3. Execute phased global rollout by region

Appendix 5: Gantt Chart - Initial versus Endpoint

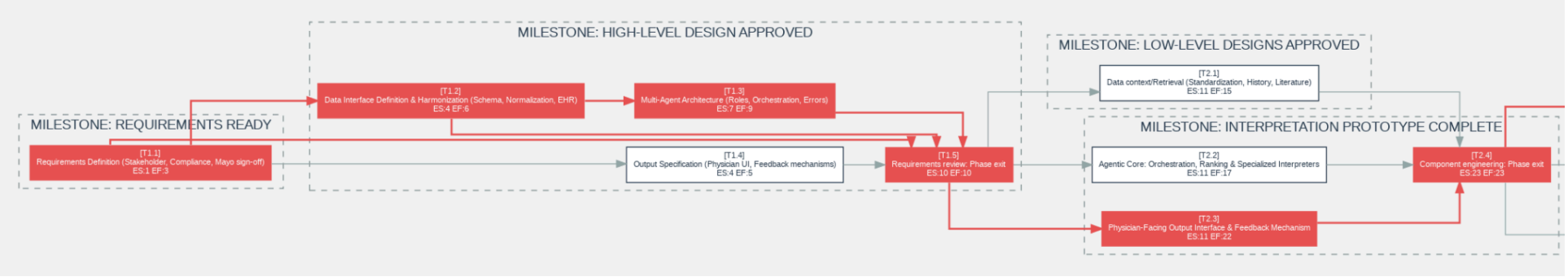
Comparison of the generated Gantt charts from our initial (first plot) variation in our tradespace walk versus the endpoint using the strategic scaling (second plot)

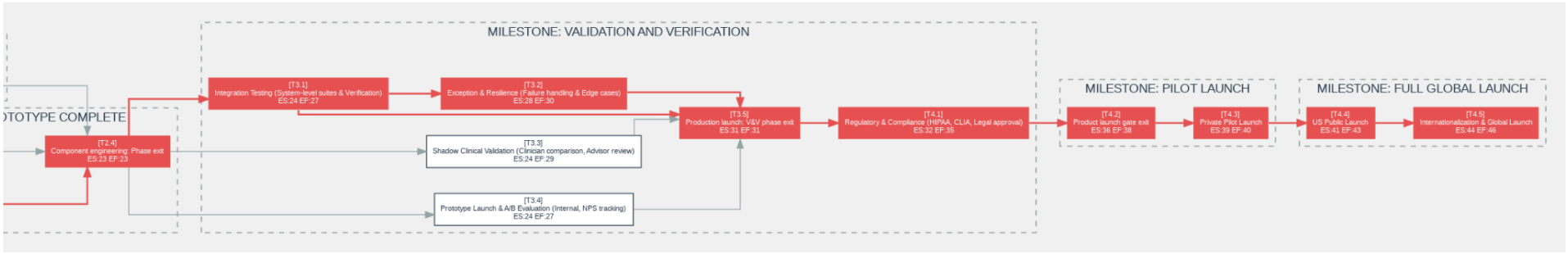


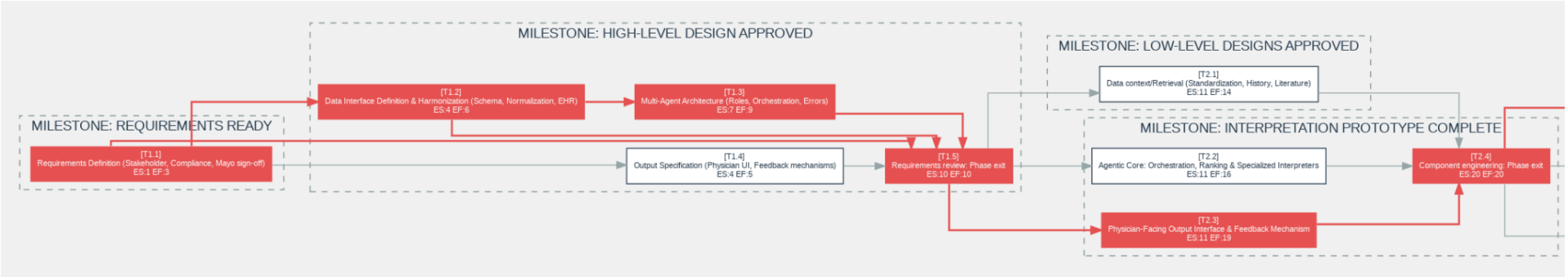


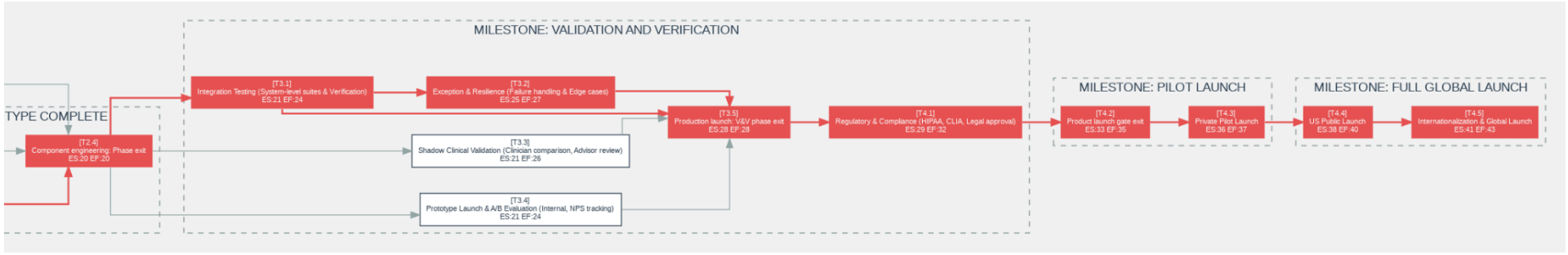
Appendix 6: CPM - Initial versus Endpoint

The Critical Path Method (CPM) is also part of our model output and can further illustrate the WBS with some additional understanding. The top shows the CPM for the initial state in two parts for formatting. The next two sections are the strategic scaled endpoint for comparison, also in two parts for formatting.









Appendix 7: References

[1] U.S. Bureau of Labor Statistics, “Software Developers, Quality Assurance Analysts, and Testers,” Occupational Outlook Handbook, 2025. [Online]. Available: https://www.bls.gov/ooh/computer-and-information-technology/software-developers.htm. [Accessed: Apr. 12, 2026].

[2] U.S. Bureau of Labor Statistics, “Computer Systems Analysts,” Occupational Outlook Handbook, 2025. [Online]. Available: https://www.bls.gov/ooh/computer-and-information-technology/computer-systems-analysts.htm. [Accessed: Apr. 12, 2026].

[3] U.S. Bureau of Labor Statistics, “Data Scientists,” Occupational Outlook Handbook, 2025. [Online]. Available: https://www.bls.gov/ooh/math/data-scientists.htm. [Accessed: Apr. 12, 2026].

[4] U.S. Bureau of Labor Statistics, “Web Developers and Digital Designers,” *Occupational Outlook Handbook*, 2025. [Online]. Available: https://www.bls.gov/ooh/computer-and-information-technology/web-developers.htm. [Accessed: Apr. 12, 2026].

[5] U.S. Bureau of Labor Statistics, “Database Administrators and Architects,” *Occupational Outlook Handbook*, 2025. [Online]. Available: https://www.bls.gov/ooh/computer-and-information-technology/database-administrators.htm. [Accessed: Apr. 12, 2026].

[6] U.S. Bureau of Labor Statistics, “Project Management Specialists,” *Occupational Outlook Handbook*, 2025. [Online]. Available: https://www.bls.gov/ooh/business-and-financial/project-management-specialists.htm. [Accessed: Apr. 12, 2026].

[7] Mercer, “Workforce Turnover Trends,” iMercer, 2024–2025. [Online]. Available: https://www.imercer.com/articleinsights/workforce-turnover-trends. [Accessed: Apr. 19, 2026].

[8] F. P. Brooks, Jr., *The Mythical Man-Month: Essays on Software Engineering*, Anniversary ed. Reading, MA, USA: Addison-Wesley, 1995.

Appendix 8: AI Disclosure

ChatGPT was used to help refine wording and structure and to check consistency between the simulation variants and the written narrative. AI was used to aid generating code and interpretation of code written by other team members, but the vast majority of the code and tests was written by hand. Google’s Gemini was used to review drafts of this assignment and act as the role of a MITsdm TA providing feedback on sections with strong arguments and areas requiring improvement. All work in response to this feedback along with all modeling choices, inputs, outputs, and conclusions are the team’s own.